

React Router DOM - Complete Documentation (Hinglish)

Contents

React Router DOM Documentation (Hinglish Mein)	1
1. Introduction	1
2. Installation	2
3. Basic Setup	2
4. Routes aur Route Component	2
5. Navigation - Link aur NavLink	3
6. Dynamic Routes aur useParams	4
7. useNavigate Hook	4
8. Nested Routes	4
9. useSearchParams (Query Parameters)	5
10. Protected Routes (Private Routes)	6
11. Common Hooks Summary	6
12. Common Mistakes (Errors jo aksar hoti hain)	6
13. Conclusion	7

React Router DOM Documentation (Hinglish Mein)

1. Introduction

React Router DOM ek library hai jo React applications mein **routing** ka kaam karti hai. Matlab, agar aapko apni app mein multiple pages (jaise Home, About, Contact) banane hain bina page reload kiye, to React Router DOM use hota hai. Ye Single Page Application (SPA) banane mein help karta hai jahan URL change hone par sirf component change hota hai, poora page reload nahi hota.

Kyun use karte hain?

- Fast navigation (no full page reload)
- Clean URL structure
- Nested routes support karta hai
- Dynamic parameters (jaise /user/:id) handle karta hai
- Protected/Private routes banane mein help karta hai

2. Installation

Sabse pehle apne React project mein React Router DOM install karna padega:

```
npm install react-router-dom
```

Ya phir yarn use karte hain to:

```
yarn add react-router-dom
```

Version check karne ke liye:

```
npm list react-router-dom
```

3. Basic Setup

Sabse pehle main.jsx ya index.js file mein BrowserRouter se poori App ko wrap karna hota hai. Ye router context provide karta hai puri application ko.

```
// main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

BrowserRouter HTML5 history API use karta hai jisse clean URLs milte hain (jaise /about, /contact).

4. Routes aur Route Component

Ab App.jsx mein Routes aur Route components use karke apne paths define karenge.

```
// App.jsx
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";
import NotFound from "./pages/NotFound";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  );
}
```

```

    <Route path="*" element={<NotFound />} />
  </Routes>
);
}

export default App;

```

Important points:

- Routes ek wrapper hai jo saare Route ko andar rakhta hai
- path="*" ka matlab hai koi bhi undefined route (404 page ke liye use hota hai)
- element prop mein JSX pass karte hain, jo route match hone par render hoga

5. Navigation - Link aur NavLink

Normal <a> tag use karne se page reload ho jata hai, isliye React Router Link component deta hai.

```

import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}

```

NavLink ek special version hai jo active route ko style karne mein madad karta hai:

```

import { NavLink } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <NavLink
        to="/about"
        className={({ isActive }) => (isActive ? "active-link" : "")}
      >
        About
      </NavLink>
    </nav>
  );
}

```

isActive boolean value deta hai jisse pata chalta hai current route active hai ya nahi.

6. Dynamic Routes aur useParams

Kabhi kabhi hume dynamic data ke basis par route banana hota hai, jaise user profile page /user/123.

```
<Route path="/user/:userId" element={<UserProfile />} />
```

Ab UserProfile component mein useParams hook se ye userId nikal sakte hain:

```
import { useParams } from "react-router-dom";

function UserProfile() {
  const { userId } = useParams();

  return <h2>User ID: {userId}</h2>;
}
```

Agar URL hai /user/123 to userId ki value "123" milegi (string ke roop mein).

7. useNavigate Hook

useNavigate hook programmatically navigate karne ke liye use hota hai, jaise form submit hone ke baad redirect karna.

```
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // login logic yahan
    navigate("/dashboard");
  };

  return <button onClick={handleLogin}>Login</button>;
}
```

Back jaane ke liye:

```
navigate(-1); // browser back button jaisa
```

Replace karke navigate karna (history stack mein add nahi hoga):

```
navigate("/login", { replace: true });
```

8. Nested Routes

Nested routes tab use hote hain jab ek page ke andar sub-pages ho, jaise Dashboard ke andar Profile aur Settings.

```

<Routes>
  <Route path="/dashboard" element={<Dashboard />}>
    <Route path="profile" element={<Profile />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Routes>

```

Dashboard component ke andar Outlet use karna padega jahan nested route render hoga:

```

import { Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <Outlet />
    </div>
  );
}

```

Jab URL /dashboard/profile hoga, tab Profile component Outlet ki jagah render hoga.

9. useSearchParams (Query Parameters)

Agar URL mein query params hain, jaise /products?category=shoes, to useSearchParams use karte hain.

```

import { useSearchParams } from "react-router-dom";

function Products() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get("category");

  return (
    <div>
      <p>Selected category: {category}</p>
      <button onClick={() => setSearchParams({ category: "bags" })}>
        Show Bags
      </button>
    </div>
  );
}

```

10. Protected Routes (Private Routes)

Agar koi route sirf logged-in users ke liye accessible hona chahiye, to ek wrapper component banate hain.

```
import { Navigate, Outlet } from "react-router-dom";

function ProtectedRoute({ isAuthenticated }) {
  return isAuthenticated ? <Outlet /> : <Navigate to="/login" />;
}
```

Isko routes mein aise use karenge:

```
<Routes>
  <Route path="/login" element={<Login />} />

  <Route element={<ProtectedRoute isAuthenticated={userIsLoggedIn} />>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/profile" element={<Profile />} />
  </Route>
</Routes>
```

Agar user authenticated nahi hai, to automatically /login par redirect ho jayega.

11. Common Hooks Summary

Hook	Kaam
useParams()	URL se dynamic parameters nikalna
useNavigate()	Programmatically navigate karna
useLocation()	Current URL/location object access karna
useSearchParams()	Query string params read/write karna
useOutletContext()	Parent route se nested route ko data pass karna

useLocation example:

```
import { useLocation } from "react-router-dom";

function CurrentPage() {
  const location = useLocation();
  console.log(location.pathname); // e.g. "/about"
  return <p>Aap is page par hain: {location.pathname}</p>;
}
```

12. Common Mistakes (Errors jo aksar hoti hain)

1. **BrowserRouter wrap na karna** - App ko <BrowserRouter> se wrap karna bhool jaana, jisse "useNavigate must be used within Router" jaisi error aati hai.

2. **element ke jagah component pass karna** - Purane React Router (v5) mein `component={Home}` hota tha, lekin v6 mein `element={<Home />}` use hota hai.
3. **Outlet lagana bhool jaana** - Nested routes ke liye parent component mein `<Outlet />` add karna zaroori hai, warna child route render nahi hoga.
4. **Case sensitivity** - Path spelling galat hone par route match nahi hota.

13. Conclusion

React Router DOM React applications mein routing implement karne ka sabse popular tarika hai. Isse hum multiple pages, dynamic routes, nested routes aur protected routes easily manage kar sakte hain, wo bhi bina page reload kiye. Practice karne se ye concepts aur clear ho jayenge - khud ek chhota project banake try karo!